

TENANTLY

A Mobile Property Management System for Small Landlords and Tenants

PROJECT REPORT

Submitted by: [STUDENT NAME]

Register Number: [REGISTER NUMBER]

Under the Guidance of: [GUIDE NAME]

Department of Computer Science

[UNIVERSITY NAME]

August 2026

CERTIFICATE

This is to certify that the project report entitled ‘Tenantly: A Mobile Property Management System for Small Landlords and Tenants’ is a record of work carried out by [STUDENT NAME] under the guidance of [GUIDE NAME] in partial fulfilment of the requirements for the award of [DEGREE NAME].

The work presented in this report has not been submitted, either in full or in part, for any other academic award.

Project Guide: _____

Head of Department:

DECLARATION

I hereby declare that this project report is my original work and that all sources of information used in its preparation have been acknowledged in the reference section. The Tenantly application and accompanying documentation were prepared for academic evaluation.

Signature of Student: _____

Date:

AI USAGE DECLARATION

Artificial intelligence tools were used as writing and formatting assistance during the preparation of this report. The project design, repository analysis, technical claims, test results and implementation descriptions were reviewed against the Tenantly source code, migrations and verification output. No AI-generated statement has been used as evidence in place of project artefacts or actual test results.

ACKNOWLEDGEMENTS

I express my sincere gratitude to [GUIDE NAME] for continuous guidance, timely feedback and encouragement throughout the project. I thank the faculty members of the Department of Computer Science, [UNIVERSITY NAME], for creating the academic environment that made this work possible.

I am grateful to my classmates, friends and family members for their support during the analysis, development, testing and documentation of Tenantly. Finally, I acknowledge everyone who directly or indirectly contributed to the completion of this project.

ABSTRACT

Tenantly was developed as a cross-platform mobile property-management system for small landlords, paying guests, hostel operators and tenants in India. The project addressed the operational fragmentation created by spreadsheets, handwritten registers and disconnected messaging channels. The implemented application consolidated property inventory, residents, tenancy assignments, invoices, manual payment verification, receipts, complaints, notices, documents and operational reporting into role-aware workflows. The client application was built with Expo, React Native and TypeScript, while Supabase provided authentication, PostgreSQL persistence, Row Level Security, private storage and database functions. The design deliberately treats organization membership as an authorization boundary and places financial and occupancy mutations inside transactional database operations. This reduces the risk of duplicate invoices, overlapping occupancy assignments and unauthorized payment decisions. The project was verified through TypeScript compilation, linting and a Jest suite comprising forty passing tests across sixteen suites. The report documents the requirements, architecture, schema, implementation approach, security model and test evidence. Tenantly demonstrates that a mobile-first system can give small property operators a clearer operational view while preserving tenant privacy and auditable financial records.

Keywords: property management, React Native, Expo, Supabase, Row Level Security, tenancy, invoices, mobile application.

TABLE OF CONTENTS

CHAPTER 1	2
1.1 Overview of Tenantly	2
1.2 Background and Motivation	2
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope and Applicability	3
1.6 Intended Users	3
1.7 Organization of the Report.....	3
CHAPTER 2	4
2.1 Existing Property-Management Practices.....	4
2.2 Limitations of the Existing Process	4
2.3 Literature Review.....	4
2.4 Proposed System.....	4
2.5 Benefits of Tenantly.....	5
2.6 Functional Requirements	5
2.7 Non-Functional Requirements	5
2.8 Software and Hardware Requirements	6
2.9 Feasibility Analysis.....	6
CHAPTER 3	7
3.1 Overall Architecture.....	7
3.2 Role and Use-Case Design.....	7
3.3 Module Design.....	8
3.4 Data-Flow Design	8
3.5 Entity-Relationship Design.....	9
3.6 Core Database Entities.....	10
3.7 Authorization and Security Design.....	10

3.8 Workflow and Navigation Design	11
CHAPTER 4	12
4.1 Development Methodology	12
4.2 Technology Stack.....	12
4.3 Authentication and Protected Navigation	12
4.4 Organization, Property and Occupancy Management	12
4.5 Billing, Payments and Receipts	12
4.6 Complaints, Notices, Documents and Operations	13
4.7 Dashboards, Reports and Shared Components	13
4.8 Database Migrations and RLS	13
4.9 Validation, Errors and Reliability	13
4.10 Application Interface Screens	14
CHAPTER 5	30
5.1 Testing Objectives and Environment.....	30
5.2 Unit and Component Testing.....	30
5.3 Static Analysis	30
5.4 Database and RLS Test Readiness.....	30
5.5 Expo Environment Check.....	31
5.6 Test Execution Summary.....	31
5.7 Representative Test Cases.....	31
5.8 Defect and Risk Resolution	32
CHAPTER 6	33
6.1 Summary of Implementation	33
6.2 Achievement of Objectives.....	33
6.3 Advantages.....	33
6.4 Current Limitations	33
6.5 Future Scope	34

Appendix A: Installation and Configuration.....	36
Configuration Checklist.....	36
Appendix B: Basic User Manual	37
Owner and Manager Workflow	37
Tenant Workflow	37
Maintenance Staff Workflow.....	37
Appendix C: Important Database Functions.....	38
Data Integrity Controls	38
Appendix D: Extended Test-Case Matrix.....	39
Appendix D Continued: Extended Test-Case Matrix	40
Appendix E: Verification Commands.....	41
Appendix F: Screenshot Finalization Checklist.....	41

LIST OF FIGURES

Figure 1. Tenantly system architecture

Figure 2. Role-aware use-case model

Figure 3. Core data model

Figure 4. Monthly invoice and payment workflow

Figure 5. Complaint resolution workflow

Figure 6. Role-aware navigation

Figure 7. Application interface screenshots: Figures 4.1-4.16

LIST OF TABLES

Table 1. Functional requirements

Table 2. Non-functional requirements

Table 3. Technology stack

Table 4. Core data entities

Table 5. Test execution summary

Table 6. Representative test cases

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CSV	Comma-Separated Values
RLS	Row Level Security
RPC	Remote Procedure Call
SDK	Software Development Kit
UI	User Interface
UUID	Universally Unique Identifier

CHAPTER 1

INTRODUCTION

1.1 Overview of Tenantly

Tenantly is a mobile property-management application designed for small landlords, paying-guest owners, hostel managers, tenants and maintenance staff. It was implemented as a private operational system rather than a public rental marketplace. The application organizes the recurring activities of property inventory, resident onboarding, tenancy terms, occupancy, billing, payment verification, complaints, notices and documents within a single role-aware environment.

1.2 Background and Motivation

Small property operators frequently manage rent, occupancy and maintenance through paper registers, spreadsheet files and messaging applications. These methods are flexible at a very small scale but become difficult to audit when a property contains multiple rooms, beds, residents and billing periods. Tenantly was motivated by the need for an accessible mobile workflow that reduces duplicate entries, makes current occupancy visible and preserves a traceable record of financial and operational decisions.

1.3 Problem Statement

The core problem was the absence of a simple mobile system that combines property operations with tenant-facing self-service while enforcing privacy between organizations. Manual records do not reliably represent room transfers, partial payments, payment approvals or complaint histories. A system was required to provide an authoritative view of each entity without exposing another organization's information.

1.4 Objectives

The project objectives were to implement secure sign-in and organization membership; manage properties, rooms, beds, residents and tenancies; preserve occupancy history; generate monthly invoices idempotently; collect and approve

manual payment evidence; issue immutable receipts; manage complaints, notices and private documents; and provide role-specific dashboards and exports.

1.5 Scope and Applicability

The implemented MVP is applicable to small residential properties, PG accommodation and hostel-like operations. The initial release prioritizes trustworthy manual payment recording over payment-gateway integration. It supports owner, manager, tenant and maintenance-staff workflows within an organization. Public property discovery, online payment gateways and broad marketplace features were intentionally kept outside the implemented scope.

1.6 Intended Users

Owners and managers use Tenantly to configure operations, review finance and handle resident requests. Tenants use the application to view invoices, submit payment proof, access notices and create complaints. Maintenance staff receive and update assigned work. This separation makes the user interface focused while the database maintains the authorization boundary.

1.7 Organization of the Report

The remaining chapters describe system analysis and requirements, the proposed architecture and database design, the implementation of the application and its security controls, verification evidence, and a conclusion with clearly separated future enhancements.

CHAPTER 2

SYSTEM ANALYSIS AND REQUIREMENTS

2.1 Existing Property-Management Practices

The existing process generally distributes information across spreadsheets, paper documents and message threads. A room may be available in one register while a separate chat records an informal booking. Rent entries can be updated independently of occupancy changes, which makes reconciliation difficult. The process also provides limited tenant self-service because residents must contact an owner for routine information such as outstanding rent or the latest notice.

2.2 Limitations of the Existing Process

Manual and disconnected approaches create duplicate data entry, unclear ownership of records, delayed communication and weak auditability. They do not reliably enforce exclusivity when assigning beds, provide no consistent workflow for payment approval, and rarely isolate information by property organization. Tenantly addresses these gaps through a single backend model, transaction-oriented operations and role-based navigation.

2.3 Literature Review

The design draws on established information-system principles: normalized relational data for operational consistency, role-based access for controlled data sharing, mobile interaction patterns for field use, and transactional updates for financial records. The selected implementation also follows platform guidance that database authorization should be enforced through Row Level Security rather than only through client-side navigation. Official documentation for Expo, React Native, Supabase and PostgreSQL was used to validate the selected technology approach.

2.4 Proposed System

The proposed system is a cross-platform client application backed by Supabase. Users authenticate before accessing role-protected routes. Properties, occupancy and finance are managed in PostgreSQL. Private document paths are stored in the database and accessed through protected storage policies. Sensitive

multi-row actions, such as creating a tenancy with an assignment or deciding a payment, are placed behind database functions so business rules remain authoritative even if a client is retried.

2.5 Benefits of Tenantly

The system centralizes operational data, reduces repeated manual calculations, enables tenant self-service, captures approval history and improves visibility of vacancies and outstanding rent. The use of organization-scoped records and RLS policies supports privacy across property operators. The mobile-first experience also makes routine updates practical during on-site property work.

2.6 Functional Requirements

ID	Requirement	Implemented evidence
FR-01	Users can register, sign in, reset credentials and use protected routes.	Auth route group and session-aware root layout.
FR-02	Owners can create organizations, properties, rooms, beds and residents.	Organization, property, room and resident setup routes.
FR-03	Tenancies preserve assignment history and prevent conflicting active occupancy.	Transactional tenancy and transfer functions.
FR-04	Owners can generate invoices and manage manual payments.	Billing services, invoice routes and payment functions.
FR-05	Tenants can submit proof; authorized users decide the payment.	Payment detail routes and RLS-protected decision function.
FR-06	Users can create, view and transition complaints.	Complaint routes, events and transition function.
FR-07	Owners publish targeted notices; recipients maintain read state.	Notice services, targets and read records.
FR-08	Users access role-specific dashboards and reports.	Owner, tenant and staff route groups; CSV export.

2.7 Non-Functional Requirements

Category	Requirement	Design response
Security	Organizations must not access one another's records.	RLS policies and organization IDs on business records.
Integrity	Financial and occupancy operations must	Database functions, constraints,

	be auditable.	allocations and receipt immutability.
Usability	First-time owners should complete setup without training.	Dedicated setup screens and focused navigation.
Reliability	Repeated mobile requests must not create duplicate results.	Idempotent invoice generation and transaction-oriented mutations.
Accessibility	The interface must support clear labels and usable targets.	Shared component system and platform-aware controls.
Maintainability	The system should separate domain logic from routes.	Feature-oriented services, shared utilities and generated database types.

2.8 Software and Hardware Requirements

Component	Requirement
Development environment	Node.js, npm, Expo CLI and Supabase CLI
Mobile client	Android 7+ or iOS 16.4+ baseline
Backend	Supabase Auth, PostgreSQL, Storage and Edge/database services
Recommended workstation	Modern dual-core CPU, 8 GB RAM, broadband internet
Testing	Jest, TypeScript compiler, ESLint and Supabase pgTAP suite

2.9 Feasibility Analysis

Tenantly was technically feasible because the selected stack supports a shared JavaScript and TypeScript codebase for Android, iOS and web. Operational feasibility was supported by simple task-oriented screens and by retaining manual payment verification, which matches common small-property practice. Economic feasibility was improved by managed backend services and a single client codebase. The project's main dependencies are internet availability and the correct operational configuration of Supabase policies and storage.

CHAPTER 3

SYSTEM DESIGN

3.1 Overall Architecture

Tenantly uses a layered architecture. The Expo application presents role-aware workflows and uses shared feature services for data access. Supabase Auth provides authenticated identity, while PostgreSQL stores operational records. Row Level Security applies row-level authorization, private storage holds evidence and documents, and database functions handle operations that must be atomic.

Tenantly system architecture

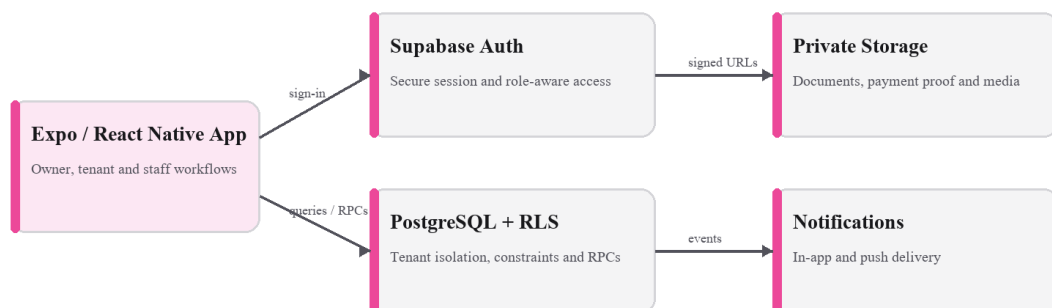


Figure 3.1. Tenantly system architecture.

3.2 Role and Use-Case Design

The model separates interface access by role while keeping membership data in the backend. An owner or manager has operational control within an authorized organization. A tenant can access only records connected to their own tenancy. Maintenance staff have narrowly scoped task access.

Role-aware use-case model

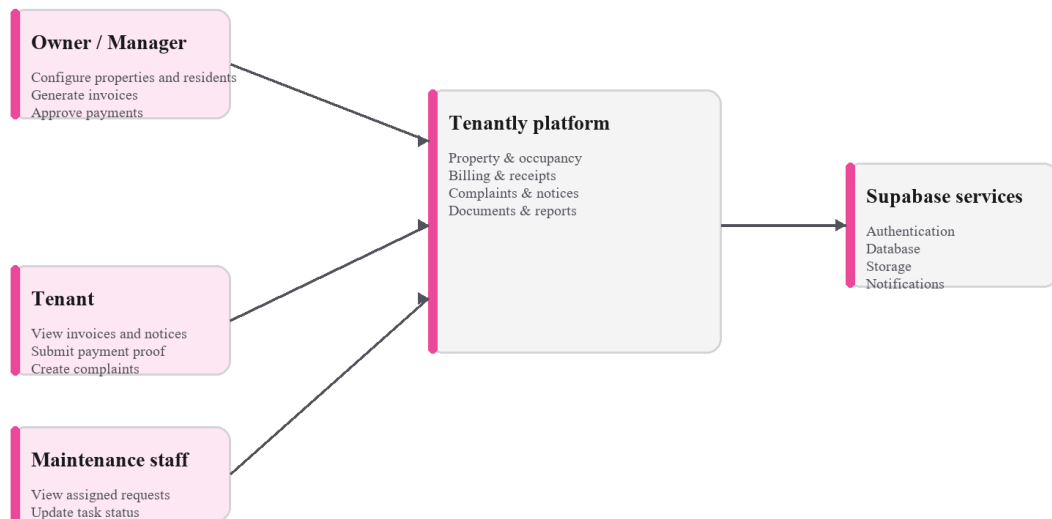


Figure 3.2. Role-aware use-case model.

3.3 Module Design

The application is divided into authentication, organizations, properties, residents, billing, complaints, notices, operations, notifications and shared infrastructure. This feature-oriented structure keeps query logic and domain rules close to the screens that use them while centralizing cross-cutting concerns such as error translation, query keys, money formatting and session state.

3.4 Data-Flow Design

For a protected action, the client submits an authenticated request to an approved database operation. The operation validates organization membership, domain state and supplied values before committing changes. The client then invalidates or refreshes only the affected query data. This design ensures that route visibility does not become the only access-control mechanism.

Monthly invoice and payment workflow

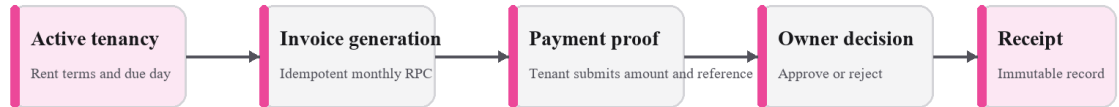


Figure 3.3. Monthly invoice and payment workflow.

3.5 Entity-Relationship Design

Each business entity carries an organization identifier so that ownership and filtering remain explicit. A resident can have a tenancy, and a tenancy produces an occupancy-assignment history. Invoices capture a billing period and may receive multiple payment allocations. Complaints preserve an event history. Documents and attachments store protected object paths instead of permanent public URLs.

Core data model

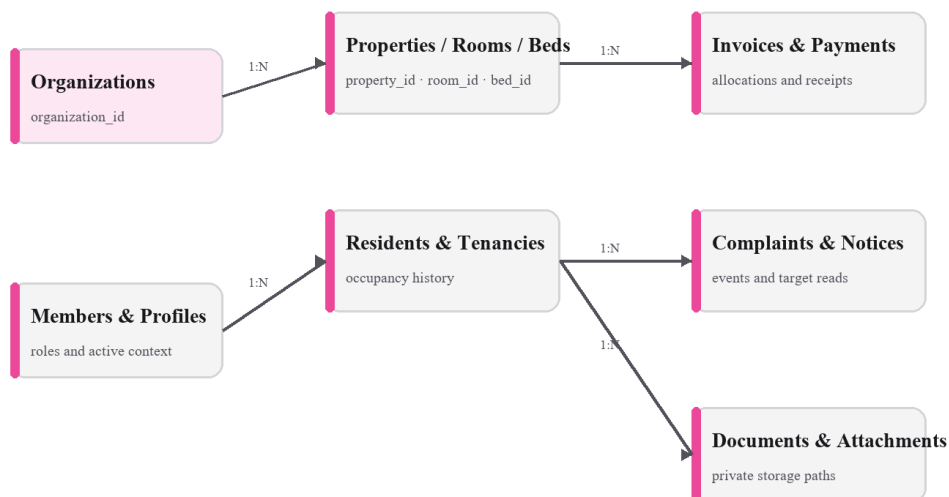


Figure 3.4. Core data model.

3.6 Core Database Entities

Entity group	Purpose
Identity and membership	Profiles, organizations, organization memberships, property memberships and invitations.
Property and occupancy	Properties, rooms, beds, residents, tenancies and occupancy assignments.
Billing	Invoices, invoice items, payments, payment allocations and receipts.
Operations	Complaints, complaint events, notices, notice targets, documents and notifications.
Audit and storage	Audit logs, attachment metadata, push devices and private object paths.

3.7 Authorization and Security Design

Authorization is enforced through PostgreSQL Row Level Security and specific database functions. Membership and property scope are checked in policies rather than inferred solely from the current screen. The mobile bundle uses only the Supabase project URL and publishable key; privileged keys are excluded. Sensitive files use private buckets and access policies. The design also uses unique constraints, foreign keys and guarded state transitions to preserve data integrity.

3.8 Workflow and Navigation Design

Complaint resolution workflow

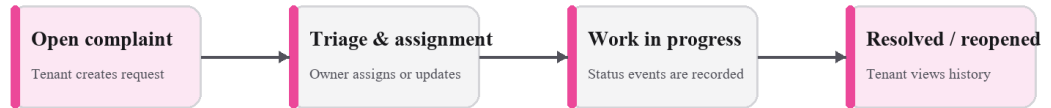


Figure 3.5. Complaint resolution workflow.

Role-aware navigation

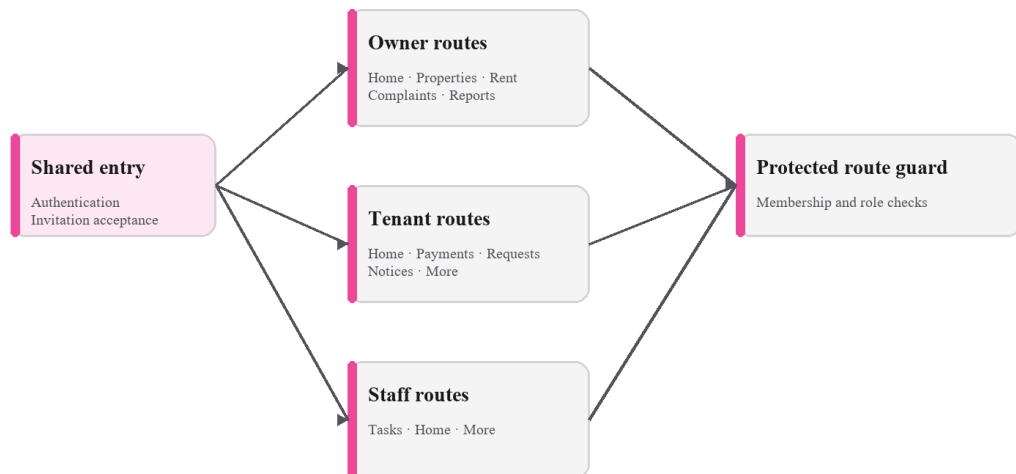


Figure 3.6. Role-aware navigation model.

CHAPTER 4

IMPLEMENTATION

4.1 Development Methodology

Tenantly was implemented iteratively. The project first established shared routing, session handling, design tokens and test tooling. It then added the database foundation and access policies before implementing property, tenancy, finance and operational features. This order reduced the risk that later screens would depend on unprotected or unstable data models.

4.2 Technology Stack

The client uses Expo SDK 56, React Native, TypeScript, Expo Router, TanStack Query, React Hook Form, Zod, Uniwind and HeroUI Native. Supabase provides authentication, PostgreSQL, private storage and policy enforcement. Jest supports unit and component testing, and the Supabase test suite is configured for database and RLS verification.

4.3 Authentication and Protected Navigation

The root layout restores the authenticated session and directs a user into public authentication routes, organization setup, invitation acceptance or an authorized application area. Route groups separate owner, tenant and staff navigation. This improves usability, but the backend remains the authority for access decisions.

4.4 Organization, Property and Occupancy Management

Owners create an organization before configuring properties, rooms and optional beds. Resident records can exist before an invite is accepted. Tenancy creation and occupancy assignment are performed through database operations that preserve history. Transfers and move-outs update active occupancy without deleting past assignments.

4.5 Billing, Payments and Receipts

Billing supports invoice line items, bulk monthly generation, manual payments and payment-proof submission. The payment-decision workflow is server-

side and produces allocations without exceeding an approved amount. Receipt creation is designed as an immutable financial record, improving traceability for both operator and resident.

4.6 Complaints, Notices, Documents and Operations

Residents can create complaints and follow status history. Operators can triage, assign and transition requests; maintenance staff receive scoped task views. Notices can be published to defined audiences and read state is maintained per recipient. Private documents, payment evidence and complaint attachments are stored through protected paths. The operational extension includes expenses, agreement renewal and a monthly profit report.

4.7 Dashboards, Reports and Shared Components

Role-specific dashboards surface information relevant to the user. The owner dashboard focuses on occupancy, collection, pending decisions and open complaints. Tenant views focus on current due amounts, active requests and notices. Shared utilities centralize money, date and CSV behavior, while shared components provide status, navigation and consistent loading, empty and error states.

4.8 Database Migrations and RLS

The repository uses timestamped SQL migrations for foundation, P0, readiness and operational updates. These migrations define tables, policies, database functions and grants together. This ensures that the client release has a versioned backend contract and that authorization is not an undocumented manual configuration.

4.9 Validation, Errors and Reliability

Client forms use typed validation and the database retains authoritative constraints. User-facing errors are normalized so a screen can explain the failure and provide a recovery path. Query keys include relevant organization scope, and high-risk financial operations avoid optimistic updates. The implementation uses idempotency and transaction boundaries where duplicate mobile requests would otherwise create inconsistent records.

4.10 Application Interface Screens

The following reserved figure areas are intentionally sized for clean screenshots captured from the final live build. Replace each area without changing its caption number or explanatory text.

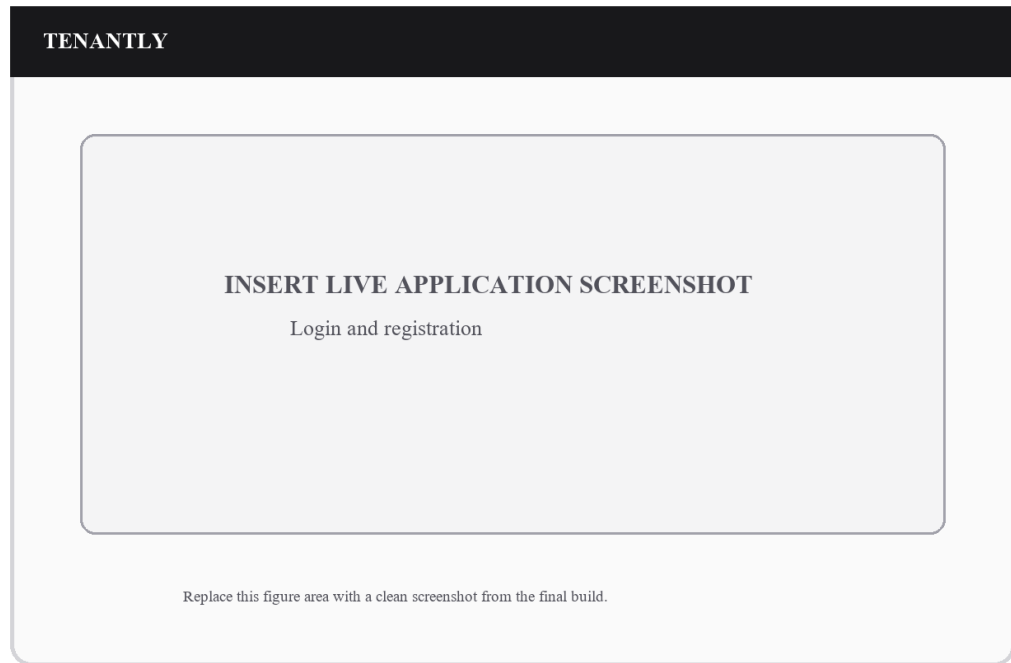


Figure 4.1. Authentication entry screen.

The authentication flow provides sign-in, registration, verification and recovery entry points.

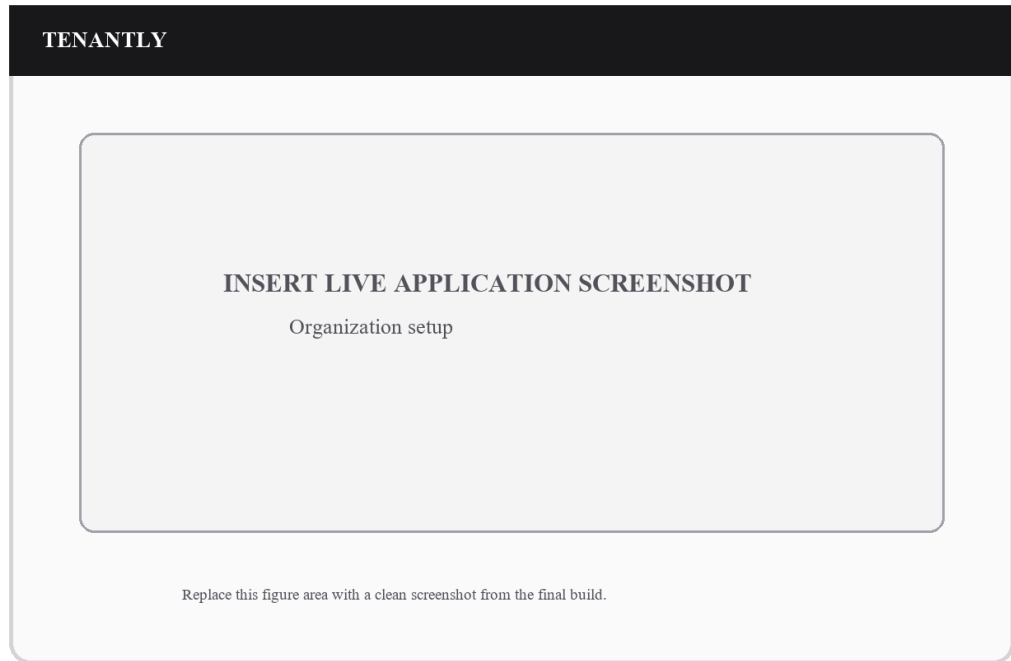


Figure 4.2. Organization setup screen.

The first-time owner workflow creates an organization and establishes the initial membership.

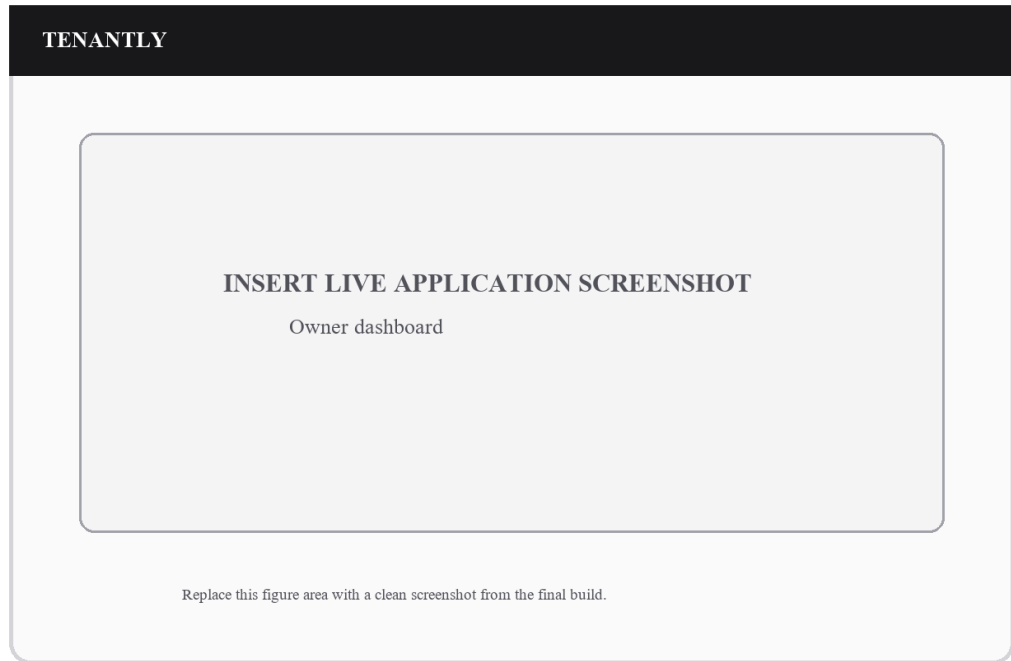


Figure 4.3. Owner dashboard.

The owner dashboard aggregates occupancy, collections, outstanding rent, approvals and complaints.

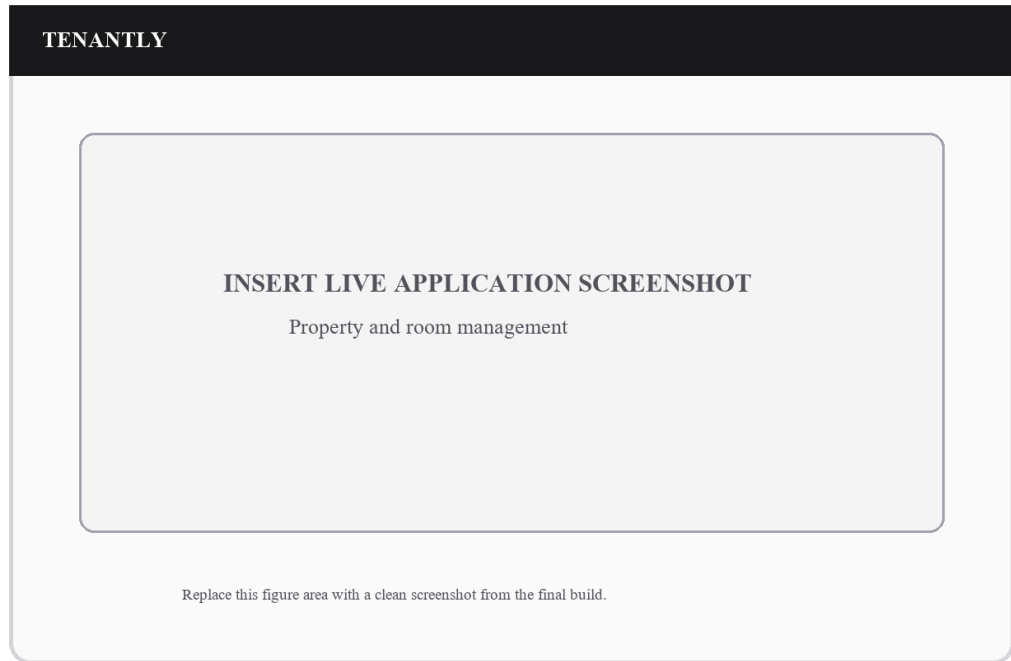


Figure 4.4. Property and room management.

Property screens allow rooms and optional beds to be configured as operational inventory.

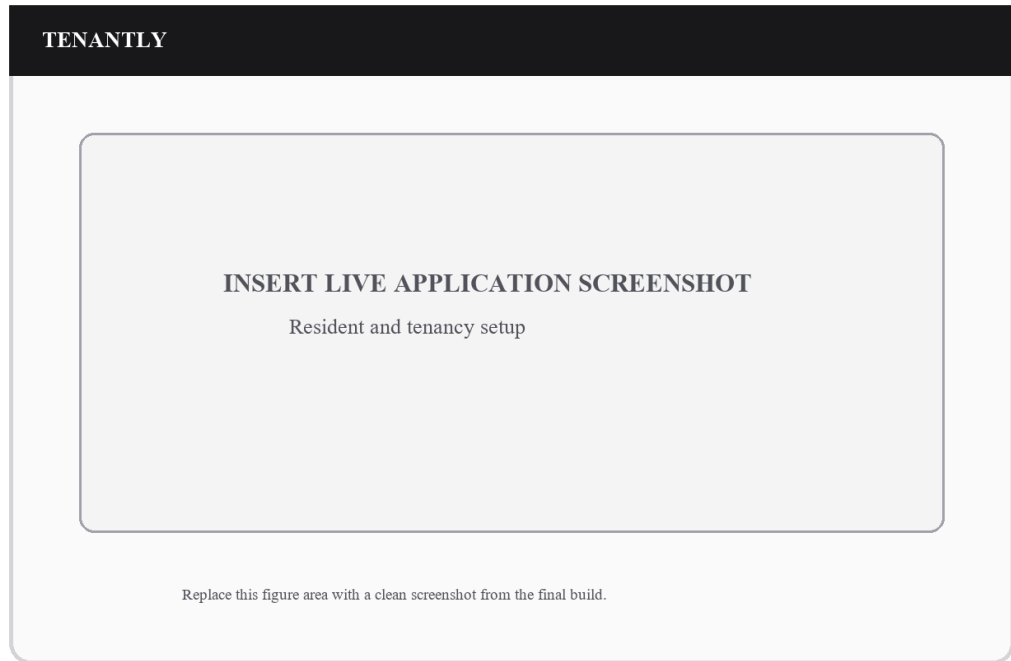


Figure 4.5. Resident and tenancy setup.

Resident records and tenancy terms are recorded before occupancy is established.

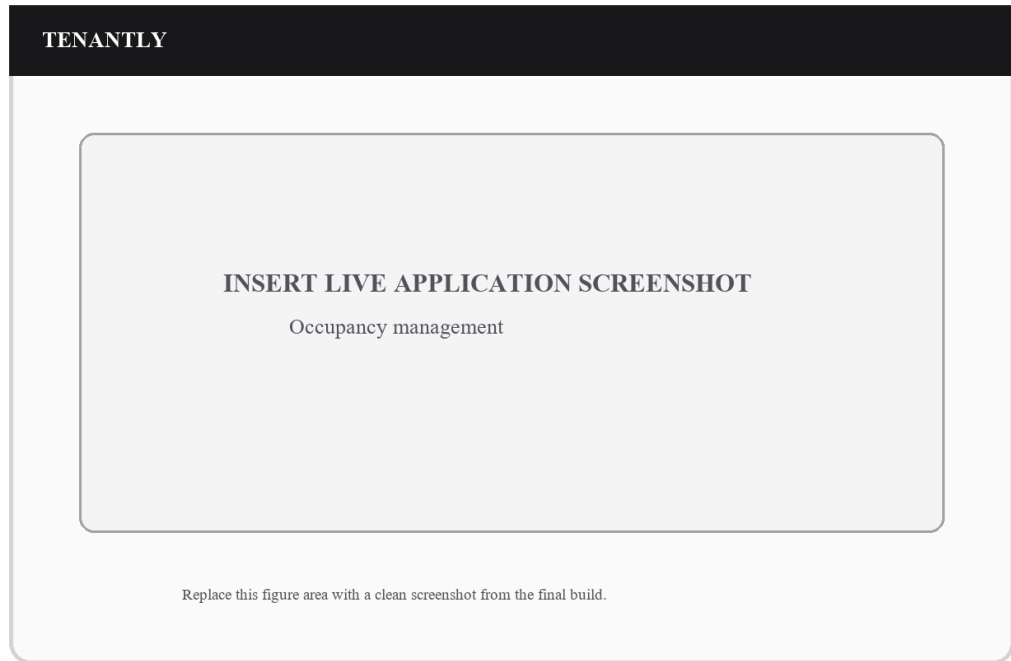


Figure 4.6. Occupancy management.

The occupancy screen supports active assignment, transfer history and vacancy derivation.

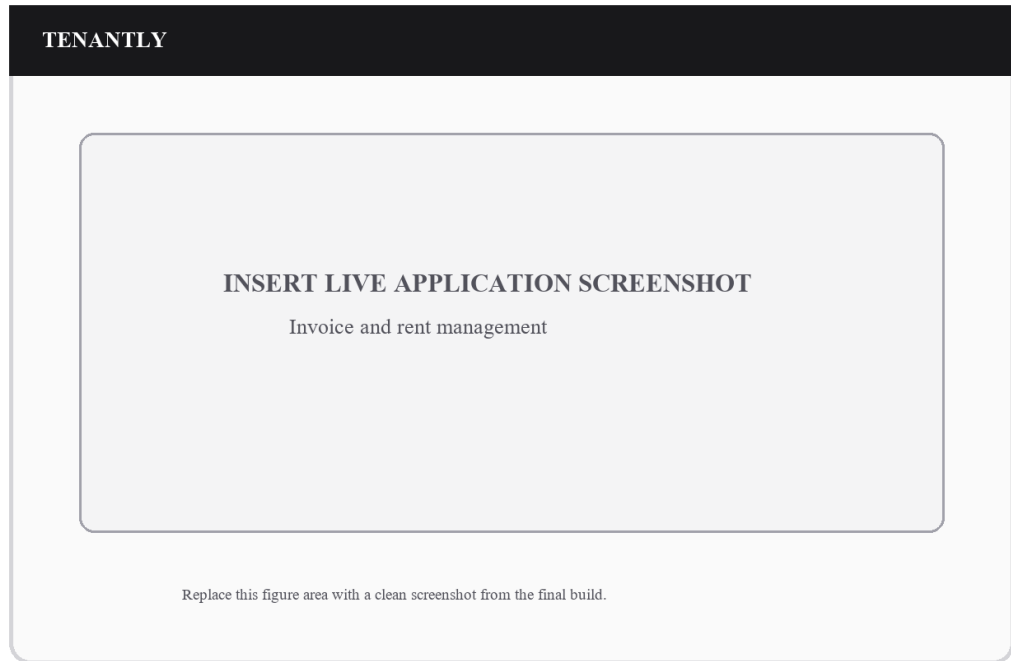


Figure 4.7. Rent and invoice management.

The rent workflow shows the invoice ledger and monthly generation actions.

TENANTLY

INSERT LIVE APPLICATION SCREENSHOT

Payment approval

Replace this figure area with a clean screenshot from the final build.

Figure 4.8. Payment approval.

An authorized operator reviews payment evidence and makes an auditable approval or rejection.

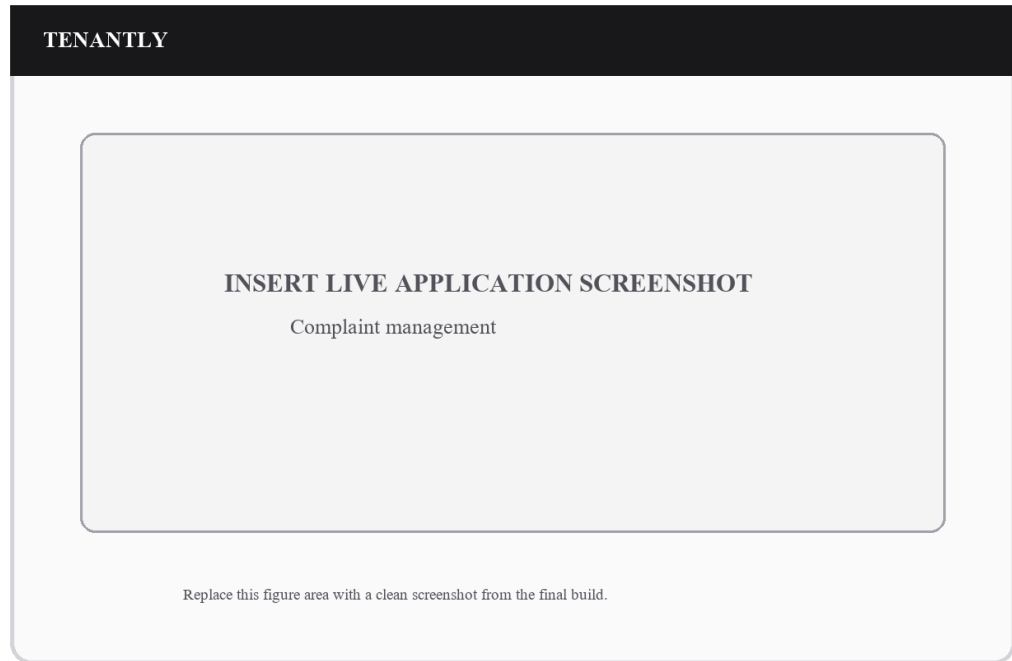


Figure 4.9. Complaint management.

Requests show priority, state, assigned staff and an event history.

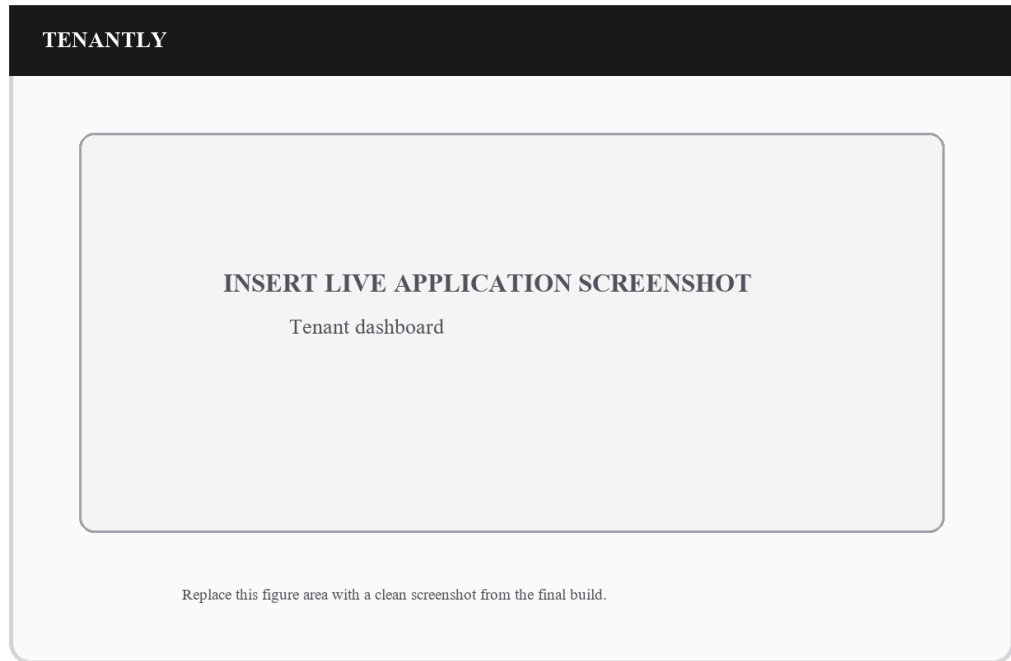


Figure 4.10. Tenant dashboard.

The tenant view provides direct access to dues, requests and notices.

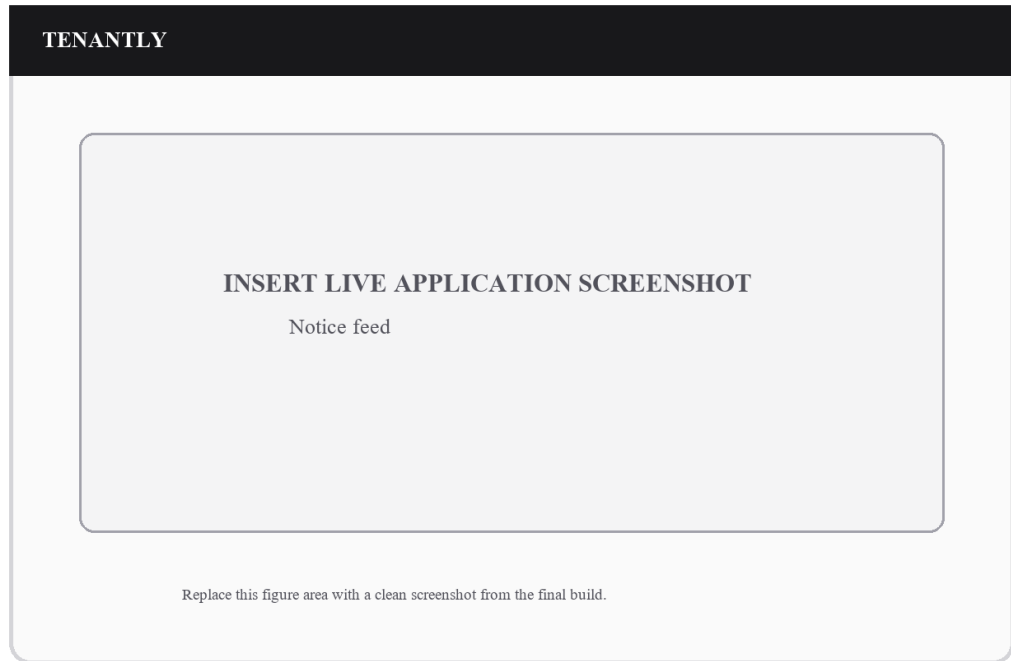


Figure 4.11. Notice feed.

Targeted notices and individual read state are visible to recipients.

SCREENSHOT SPACE: OPERATIONS AND REPORTS

Figure 4.12. Operations and reports.

Operational screens present expenses, agreements and summarized reporting.

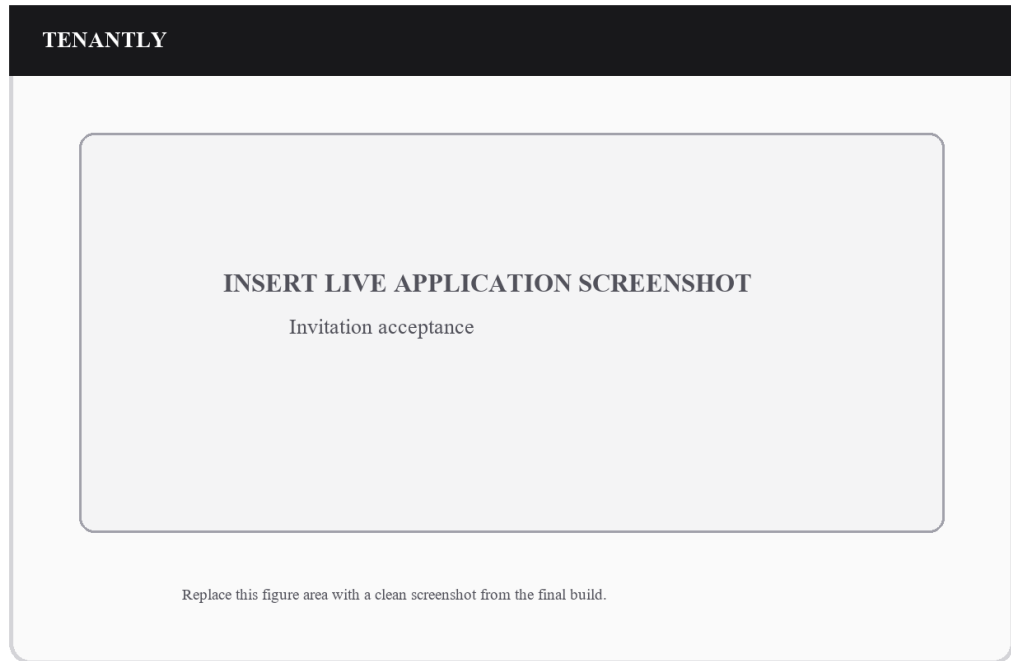


Figure 4.13. Invitation acceptance.

The invitation path joins an authorized tenant or staff user to the intended organization.

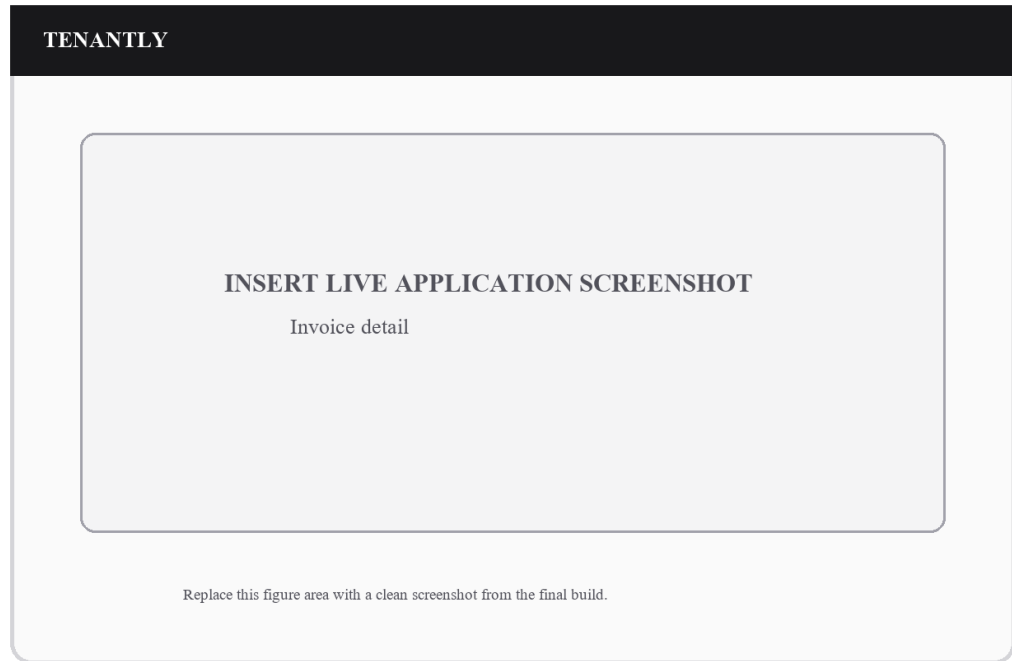


Figure 4.14. Invoice detail.

The invoice detail view presents billed line items, due state and payment history.

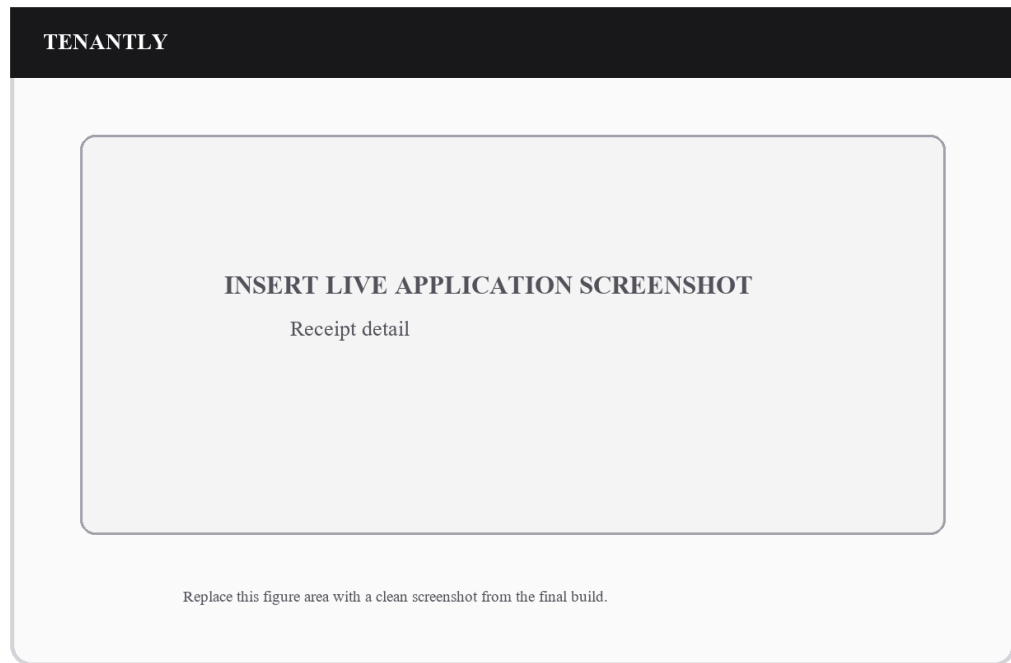


Figure 4.15. Receipt detail.

A completed payment is represented by an immutable receipt record.

TENANTLY

INSERT LIVE APPLICATION SCREENSHOT

Maintenance staff task list

Replace this figure area with a clean screenshot from the final build.

Figure 4.16. Maintenance staff task list.

Staff members see only assigned work and update its recorded operational status.

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Objectives and Environment

Testing focused on type safety, domain behavior, shared UI state, formatting utilities and database-oriented rules. The repository contains Jest tests for properties, residents, billing, complaints, operations, organization services, shared utilities and navigation components. Supabase test files are present for schema, RLS, release-readiness and demonstration operations.

5.2 Unit and Component Testing

The verification run completed successfully with 16 Jest test suites and 40 tests passing. The suite included money and date utilities, CSV export, query keys, error translation, organization services, billing aggregation, room handling, resident services, complaint services, operations services and common component states.

5.3 Static Analysis

The TypeScript typecheck completed successfully without emitting output, and the Expo lint command completed successfully with zero allowed warnings. These checks provide evidence that the checked client code is type-consistent and conforms to configured lint rules.

5.4 Database and RLS Test Readiness

The project configures a Supabase database test suite, including schema, RLS, P0 and release-readiness files. During the report verification run, the command reached the local-database connection step but could not connect because a local PostgreSQL/Supabase stack was not running in the environment. This is recorded as an environment limitation, not reported as a passed database execution. The SQL migrations and pgTAP files remain available for execution after local Supabase startup.

5.5 Expo Environment Check

Expo Doctor completed 21 of 22 checks. The remaining advisory identifies a known Hermes memory regression affecting the installed Expo SDK 56 / Hermes combination and recommends a future SDK upgrade. This advisory does not invalidate the passing type, lint or Jest checks, but it is retained as a release-risk item for the future-scope section.

5.6 Test Execution Summary

Verification activity	Observed result	Status
Jest unit/component suite	16 suites passed; 40 tests passed.	PASS
TypeScript compilation	tsc --noEmit completed successfully.	PASS
Lint	expo lint --max-warnings=0 completed successfully.	PASS
Supabase database tests	Could not connect to local PostgreSQL because local stack was not running.	ENVIRONMENT BLOCKED
Expo Doctor	21/22 checks passed; Hermes upgrade advisory reported.	ADVISORY

5.7 Representative Test Cases

ID	Scenario	Expected result	Evidence
TC-01	Money formatting	Indian currency values are formatted consistently.	Jest utility test
TC-02	Room behavior	Room data and related validation are processed correctly.	Property-room service tests
TC-03	Billing aggregates	Invoice and payment aggregate results remain correct.	Billing aggregate tests
TC-04	Complaint behavior	Complaint state/service behavior is validated.	Complaint service tests
TC-05	Operations services	Operational records are handled through tested services.	Operations service tests
TC-06	Navigation states	Shared screen navigation logic renders expected states.	Component tests
TC-07	Organization domain	Membership/organization behavior is validated.	Organization service tests

TC-08	Database isolation	Policies are supplied for organization- and tenant-scoped access.	Migration and pgTAP artefacts
-------	--------------------	---	-------------------------------

5.8 Defect and Risk Resolution

The implementation addressed high-risk scenarios through data-model choices rather than relying only on interface validation. Occupancy assignment uses transactional functions to avoid conflicts; invoice generation is designed to be idempotent; payment allocation and receipt records support auditability; and RLS policies limit organization and tenant visibility. The reported Hermes advisory remains an explicit release maintenance item rather than a concealed issue.

CHAPTER 6

CONCLUSION

6.1 Summary of Implementation

Tenantly delivered a mobile-first property-management MVP that combines property inventory, residents, tenancy history, rent operations, payment evidence, complaints, notices, private documents and reporting. The system was implemented with an emphasis on safe operational workflows rather than only interface completeness.

6.2 Achievement of Objectives

The project achieved its intended foundation: role-aware application areas, organization-scoped data, structured occupancy, invoices and payments, complaint lifecycle management, targeted communication and verification coverage for core client behavior. The architecture places financial and occupancy operations in database-level functions and supplements them with RLS policies and private storage controls.

6.3 Advantages

Tenantly improves operational clarity by preserving one source of truth for properties, residents and money. It supports tenant self-service, lowers reliance on ad-hoc message threads, records decision history and uses consistent organization boundaries. The shared Expo codebase also supports a common workflow across Android, iOS and web.

6.4 Current Limitations

The MVP intentionally does not integrate an online payment gateway or marketplace search. The local database test suite requires a running local Supabase/PostgreSQL environment for execution. In addition, the installed Expo SDK 56 dependency set has a documented Hermes upgrade advisory that should be addressed before a broader production rollout.

6.5 Future Scope

Future work can add payment gateways, scheduled reminders, richer analytics, offline mutation queues, agreement automation, messaging integrations and public rental discovery. A near-term maintenance task is upgrading the Expo/React Native dependency baseline to incorporate the Hermes regression fix identified by Expo Doctor.

REFERENCES

- Expo. (2026). Core concepts. <https://docs.expo.dev/core-concepts/>
- Expo. (2026). Expo documentation. <https://docs.expo.dev/>
- Meta. (2026). React Native documentation. <https://reactnative.dev/docs/getting-started>
- Supabase. (2026). Database overview.
<https://supabase.com/docs/guides/database/overview>
- Supabase. (2026). Row Level Security.
<https://supabase.com/docs/guides/database/postgres/row-level-security>
- Supabase. (2026). Securing your API. <https://supabase.com/docs/guides/api/securing-your-api>
- Supabase. (2026). Production checklist.
<https://supabase.com/docs/guides/deployment/going-into-prod>
- PostgreSQL Global Development Group. (2026). PostgreSQL documentation.
<https://www.postgresql.org/docs/>
- TypeScript. (2026). TypeScript handbook.
<https://www.typescriptlang.org/docs/handbook/intro.html>
- TanStack. (2026). TanStack Query documentation. <https://tanstack.com/query/latest>
- Zod. (2026). Zod documentation. <https://zod.dev/>
- Jest. (2026). Jest documentation. <https://jestjs.io/docs/getting-started>

APPENDICES

Appendix A: Installation and Configuration

Install dependencies with `npm install`. Copy the environment example into `.env` and provide the Supabase project URL and publishable key. Apply Supabase migrations in timestamp order. The mobile client must not contain a service-role key. For a local backend, start Docker Desktop, run `npm run supabase:start`, execute the migrations through `npm run supabase:reset`, and then generate matching client types with `npm run supabase:types`.

Configuration Checklist

- Provide `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY` in the application environment.
- Keep service-role credentials in server-side Supabase secrets only.
- Apply migrations in timestamp order and deploy a compatible client and database change together.
- Confirm private Storage buckets, RLS policies and allowed file types before operational use.
- Use separate local, staging and production projects to avoid testing against production data.

Appendix B: Basic User Manual

Owner and Manager Workflow

- Register or sign in, then create or select an organization.
- Create a property and configure rooms; add beds where shared occupancy is required.
- Create resident records and tenancy terms, then assign a room or bed.
- Generate monthly invoices for active tenancies and review the invoice ledger.
- Review submitted payment proof, approve or reject it with an appropriate reason, and retain the resulting receipt.
- Use complaints, notices, documents and reports to manage continuing operations.

Tenant Workflow

- Sign in through the invitation-linked account and open the tenant dashboard.
- Review the current invoice and past payment state.
- Submit manual payment proof with payment method, date, reference, amount and attachment.
- Open a complaint with category, priority and a clear description; follow the status history.
- Read targeted notices and access authorized documents from the More area.

Maintenance Staff Workflow

- Sign in using the assigned staff membership.
- Open the task list to view only authorized requests.
- Add progress notes and move a request through the allowed lifecycle.
- Coordinate resolution through the recorded complaint history rather than separate message threads.

Appendix C: Important Database Functions

Function	Purpose
create_organization_with_owner	Creates the initial organization and owner membership.
create_tenancy_with_assignment	Creates tenancy and occupancy assignment in one operation.
transfer_occupancy	Closes prior assignment and creates a new active state.
generate_monthly_invoices	Creates missing invoices for eligible tenancies.
submit_payment / decide_payment	Submits payment evidence and performs authorized decision/allocation.
transition_complaint	Records validated complaint lifecycle changes.
create_invitation / accept_invitation	Issues and accepts controlled membership invitations.
publish_notice	Creates notices and their approved targets.
owner_dashboard / monthly_profit_report	Returns organization-scoped dashboard/report data.

Data Integrity Controls

Database functions are complemented by primary keys, foreign keys, unique constraints, state checks, RLS policies and storage policies. The implementation stores money as integer paise, uses time-aware occupancy history, and relies on compensating records rather than deleting completed financial evidence. These controls support an auditable operational model.

Appendix D: Extended Test-Case Matrix

ID	Test scenario	Expected outcome	Verification layer
TC-09	Email login with valid credentials	Session is restored and protected routes become available.	Manual / Auth flow
TC-10	Password reset request	Recovery flow accepts the email and provides safe feedback.	Manual / Auth route
TC-11	Organization creation	Initial owner membership is created with the organization.	Database function
TC-12	Property creation by owner	Authorized operator can create property under the active organization.	RLS / service
TC-13	Room creation with beds	Room and optional bed inventory are created together.	Database function
TC-14	Resident created before app account	Resident record can exist without linked profile.	Schema / service
TC-15	Tenancy and initial assignment	Tenancy and occupancy state are created atomically.	Database function
TC-16	Room transfer	Prior occupancy closes and new occupancy becomes active.	Database function
TC-17	Overlapping bed occupancy	Conflicting active assignment is rejected.	Constraint / function
TC-18	Monthly invoice rerun	Existing invoice is skipped rather than duplicated.	Database function
TC-19	Partial payment	Allocated amount updates balance without exceeding payment.	Billing logic
TC-20	Unauthorized payment decision	Tenant cannot approve a submitted payment.	RLS / function
TC-21	Receipt protection	Immutable receipt cannot be silently modified.	Trigger / schema
TC-22	Tenant complaint creation	Linked tenant may create a scoped complaint.	RLS / function
TC-23	Invalid complaint transition	Disallowed status update is rejected.	Database function
TC-24	Targeted notice read	Recipient can read intended notice and update own read state.	RLS / service

Appendix D Continued: Extended Test-Case Matrix

ID	Test scenario	Expected outcome	Verification layer
TC-25	Private document access	Non-authorized users cannot read sensitive storage object.	Storage policy
TC-26	Complaint attachment	Attachment path is associated with authorized complaint record.	Storage / function
TC-27	CSV export	Export preserves expected invoice/payment columns.	Jest utility test
TC-28	Date formatting	Local date helper formats values consistently.	Jest utility test
TC-29	Money formatting	Paise values are rendered consistently for INR display.	Jest utility test
TC-30	Query-key scoping	Organization/filter scope is included in query identity.	Jest utility test
TC-31	Error translation	Database/application failures map to understandable user messages.	Jest unit test
TC-32	Loading and empty states	Shared state views render the appropriate screen state.	Component test
TC-33	Navigation guards	Navigation helpers choose the correct safe route.	Component test
TC-34	Expense recording	Authorized operational expense is persisted and reportable.	Database function
TC-35	Agreement renewal	Authorized renewal updates document/tenancy operational state.	Database function
TC-36	Monthly profit report	Organization-scoped finance summary is returned.	Database function

Appendix E: Verification Commands

- `npm test` - Runs the Jest unit and component suite.
- `npm run typecheck` - Validates TypeScript without emitting files.
- `npm run lint` - Runs configured Expo lint checks.
- `npm run supabase:start` followed by `npm run supabase:test` - Starts local services and runs database tests.
- `npm run doctor` - Runs Expo environment checks.

Appendix F: Screenshot Finalization Checklist

- Replace each Figure 4.1-4.16 placeholder with a live application screenshot at the same dimensions.
- Use consistent demo data and remove personal data, access tokens, URLs and debug overlays.
- Capture owner, tenant and staff screens after the corresponding workflow is complete.
- Retain each existing caption and explanatory paragraph so figures remain cross-referenceable.
- Regenerate the Table of Contents, List of Figures and List of Tables after replacing screenshots.